

Guaranteed Scheduling

Fair CPU sharing through measurable promises

Definition

Guaranteed scheduling is a scheduling approach where the operating system promises each user or process a fair share of CPU time, then chooses processes based on how far behind or ahead they are relative to that promise.

1. Different Scheduling Philosophy

Main Idea

Instead of only using priorities, quanta, or predicted burst lengths, guaranteed scheduling starts with a performance promise and tries to keep it.

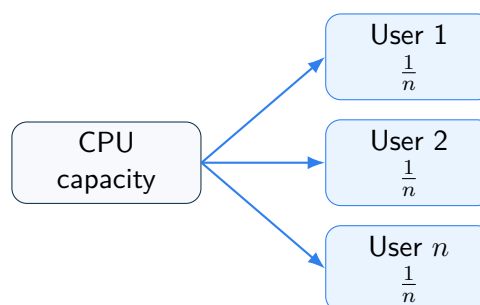
Fairness Promise

If n equal users are logged in, each user should receive about $\frac{1}{n}$ of the CPU power. Similarly, if one user has n equal runnable processes, each process should receive about $\frac{1}{n}$ of the CPU cycles.

2. What Is Being Guaranteed

Equal Share

The guarantee is not that every process finishes at the same time. The guarantee is that each equal competitor receives a fair fraction of CPU service over time.



Assumption

The simple rule assumes the users or processes are equal competitors. If the system supports different weights or priorities, the promised shares must be adjusted.

3. Information the System Tracks

Accounting Data

To keep the promise, the scheduler must know how much CPU time each process has already received since it was created.

Tracked Value	Purpose
Creation time	Tells how long the process has existed.
Elapsed time since creation	Used to compute the process's fair entitlement.
Actual CPU time consumed	Tells how much CPU service the process has already received.
Number of competitors n	Determines the fair share each process should receive.

4. CPU Entitlement

Entitled CPU Time

If a process has existed for T seconds while competing with n equal processes, then its fair CPU entitlement is:

$$\text{entitled CPU time} = \frac{T}{n}$$

Meaning

The entitlement says how much CPU time the process should have received by now if the CPU had been divided evenly among all equal competitors.

5. The Ratio

Service Ratio

The scheduler compares actual service with entitled service:

$$\text{ratio} = \frac{\text{actual CPU time consumed}}{\text{CPU time entitled}}$$

Ratio	Meaning	Scheduling Interpretation
0.5	The process has received half of what it should have received.	It is behind and should be favored.
1.0	The process has received exactly its fair share.	It is on target.
2.0	The process has received twice what it should have received.	It is ahead and can wait.

6. Scheduling Rule

Lowest Ratio Runs

Guaranteed scheduling runs the process with the lowest ratio, because that process is farthest behind its promised share.

When to Switch

The selected process runs until its ratio rises above the ratio of its closest competitor. Then the scheduler chooses the process that is now most behind.

```
for each runnable process:
    elapsed_time = current_time - creation_time
    entitled_cpu = elapsed_time / number_of_competitors
    ratio = actual_cpu_used / entitled_cpu
```

```
choose the runnable process with the lowest ratio
run it until its ratio is no longer the lowest
```

7. Snapshot Example

Four Equal Processes

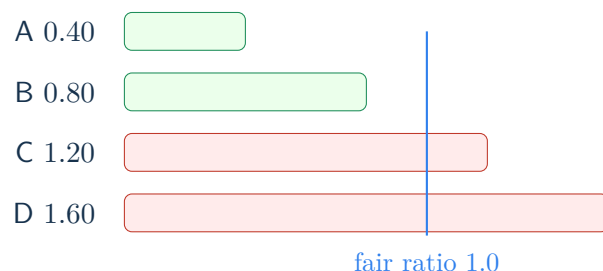
Suppose four equal processes have all existed for 100 seconds. Each process is entitled to:

$$\frac{100}{4} = 25 \text{ seconds of CPU time}$$

Process	Actual CPU Used	Entitled CPU	Ratio
A	10 s	25 s	$10/25 = 0.40$
B	20 s	25 s	$20/25 = 0.80$
C	30 s	25 s	$30/25 = 1.20$
D	40 s	25 s	$40/25 = 1.60$

Decision

Process A has the lowest ratio, so it has received the least CPU time relative to its fair share. The scheduler should run A next.



8. Running Until the Next Process Catches Up

Closest Competitor

In the snapshot, A has ratio 0.40 and B has the next lowest ratio 0.80. A should run until it catches up to B's service level.

Simplified Calculation

At the snapshot, A has used 10 seconds and B has used 20 seconds. If A runs for about 10 more seconds while B waits, A reaches 20 seconds of actual CPU use, so A is no longer clearly behind B.

Real Systems Update Continuously

In an actual implementation, elapsed time and entitlement keep changing while the process runs. The key idea is still the same: keep running the most under-served process until another process becomes more under-served.

9. User-Level vs. Process-Level Fairness

Multiple Users

On a multiuser system, the promise may be made per user. If n users are logged in, each user should receive about $\frac{1}{n}$ of the CPU power.

Single User

On a single-user system, the same idea can be applied to processes. If n equal processes are running, each process should receive about $\frac{1}{n}$ of the CPU cycles.

Reason

The same accounting method works in both cases. The scheduler compares actual CPU usage against promised CPU usage and favors the one that is behind.

10. Why This Is Fair

Fairness by Measurement

The algorithm does not merely hope that CPU time is shared fairly. It measures each process's actual CPU time and compares it with what the process should have received.

Self-Correcting Behavior

If a process has received too little CPU time, its ratio becomes low, so it is selected. If it has received too much CPU time, its ratio becomes high, so other processes are selected first.

11. Strengths and Weaknesses

Aspect	Explanation
Main benefit	Provides a clear fairness promise and a measurable way to enforce it.
Good for	Systems where users or processes should receive predictable shares of CPU time.
Required support	Accurate accounting of CPU time used by each process or user.
Main limitation	It focuses on fair CPU share, not necessarily shortest response time.
Practical issue	The number of competitors can change as users log in, processes exit, or new processes start.

12. Comparison with Other Scheduling Ideas

Algorithm	Main Rule	Main Goal
Round robin	Give each runnable process a turn.	Simple time sharing.
Priority scheduling	Run the highest-priority runnable process.	Respect importance or urgency.
Shortest Process Next	Run the process with the shortest estimated next burst.	Reduce average response time.
Guaranteed scheduling	Run the process with the lowest actual-to-entitled CPU ratio.	Keep a fair CPU-share promise.

13. Big Picture

Main Conclusion

Guaranteed scheduling is based on a simple promise: each equal competitor should receive its fair fraction of CPU time. The scheduler enforces this by tracking actual CPU usage, computing entitlement, and running the process with the lowest ratio.

Exam Shortcut

Remember the ratio:

$$\text{ratio} = \frac{\text{actual CPU time consumed}}{\text{CPU time entitled}}$$

Lowest ratio runs first. A ratio below 1 means the process is behind its fair share. A ratio above 1 means it has already received more than its fair share.

14. Quick Review

Key Points

- Guaranteed scheduling makes a measurable performance promise.
- With n equal users, each should receive about $\frac{1}{n}$ of the CPU power.
- With n equal processes, each should receive about $\frac{1}{n}$ of the CPU cycles.
- The system tracks how much CPU time each process has used since creation.
- A process's entitlement is its elapsed lifetime divided by n .
- The important ratio is actual CPU consumed divided by CPU time entitled.
- A ratio of 0.5 means the process has received only half of its fair share.
- A ratio of 2.0 means the process has received twice its fair share.
- The scheduler runs the process with the lowest ratio.
- The running process continues until its ratio rises above a close competitor's ratio.